

Lecture 16: BERT Variants

Week 6, Lecture 2 - Improvements and Optimizations

PSYC 51.07: Models of Language and Communication

Winter 2026

Today's Agenda

1.  **RoBERTa:** Robustly Optimized BERT
2.  **ALBERT:** A Lite BERT with parameter sharing
3.  **DistilBERT:** Knowledge distillation for efficiency
4.  **ELECTRA:** Replace Token Detection
5.  **Comparative Analysis:** When to use which variant
6.  **Practical Considerations:** Model selection guide

Goal: Understand improvements to BERT and choose the right model

BERT's Limitations

What could be improved?

1. Training Procedure

- Some choices seemed arbitrary
- NSP task might not be useful
- Static masking (same masks every epoch)

2. Model Size

- 110M (Base) or 340M (Large) parameters
- Large memory footprint
- Slow inference

3. Training Efficiency

- Only 15% of tokens are predicted
- 85% of computation "wasted"?
- Could we learn more efficiently?

4. Data and Compute

- Trained on limited data (3.3B words)
- Modern datasets much larger
- Could benefit from more training

Many variants address these issues!

RoBERTa: Robustly Optimized BERT

Key idea: Better training = Better performance

RoBERTa's Improvements (Liu et al. 2019):

1. Remove NSP Task

- Next Sentence Prediction hurt performance
- Use only Masked Language Modeling
- Full sentences (don't need sentence pairs)

2. Dynamic Masking

- Generate masking pattern every time
- BERT: static masks (same for every epoch)
- More diverse training signal

3. Larger Batches, More Data

- Batch size: 8K sequences (vs BERT's 256)
- 160GB text (vs BERT's 16GB)
- Longer training (500K steps vs 100K)

4. Longer Sequences

- Train with longer sequences
- Better for downstream tasks

Reference: Liu et al. (2019) - "RoBERTa: A Robustly Optimized BERT Pretraining Approach"

Consistent improvements over BERT

Task	BERT	RoBERTa	Improvement
SQuAD 1.1	90.9	94.6	+3.7
SQuAD 2.0	83.1	89.4	+6.3
MNLI	86.7	90.2	+3.5
SST-2	94.9	96.4	+1.5
RACE	72.0	83.2	+11.2

Key Findings:

- Removing NSP improves performance
- Dynamic masking better than static
- More data + longer training = better results
- RoBERTa-Large matches or beats BERT-Large on all tasks
- Sometimes huge gains (RACE: +11.2 points!)

Dynamic vs Static Masking

How masking patterns are generated

Static Masking (BERT):

Preprocessing:

- Mask tokens once
- Save masked dataset
- Same masks every epoch

Epoch 1: "My [MASK] is cute"

Epoch 2: "My [MASK] is cute"

Epoch 3: "My [MASK] is cute"

Problem:

- Model sees same masks repeatedly
- Less diverse training signal
- Potential overfitting to mask patterns

Cost: Slight computational overhead, but worth it!

Dynamic Masking (RoBERTa):

On-the-fly:

- Generate masks during training
- Different masks each time
- More variety

Epoch 1: "My [MASK] is cute"

Epoch 2: "My dog [MASK] cute"

Epoch 3: "My dog is [MASK]"

Benefits:

- More diverse training examples
- Better generalization
- Prevents memorization

Key idea: Parameter sharing for efficiency

ALBERT's Innovations (Lan et al. 2019):

1. Factorized Embedding Parameterization

- BERT: Vocabulary embedding = Hidden size ($30K \times 768$)
- ALBERT: Vocab $\square$ Small ($30K \times 128$) $\square$ Hidden (128×768)
- Saves parameters, especially for large hidden sizes

2. Cross-Layer Parameter Sharing

- Share all parameters across layers
- Same attention and FFN weights for all layers
- **Massive parameter reduction!**

3. Sentence Order Prediction (SOP)

- Replace NSP with SOP
- Predict if two sentences are in correct order
- More challenging task than NSP

Reference: Lan et al. (2019) - "ALBERT: A Lite BERT for Self-supervised Learning"

ALBERT Parameter Efficiency

Dramatic parameter reduction!

Model	Layers	Hidden	Parameters
BERT-base	12	768	110M
ALBERT-base	12	768	12M
BERT-large	24	1024	340M
ALBERT-large	24	1024	18M
ALBERT-xlarge	24	2048	60M
ALBERT-xxlarge	12	4096	235M

Key Observations:

- ALBERT-base: **89% fewer parameters** than BERT-base
- Can train much larger hidden sizes with same memory
- ALBERT-xxlarge: 4096 hidden dim, still only 235M params
- Trade-off: fewer params but similar computation (layer sharing)

Benefits

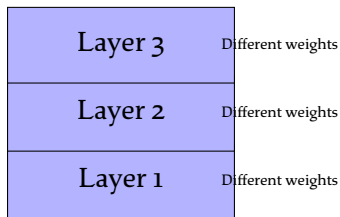
- Lower memory footprint
- Easier to deploy
- Can scale to larger hidden dimensions

Cross-Layer Parameter Sharing

How ALBERT achieves parameter efficiency

BERT (No Sharing):

separate parameters
Each layer has

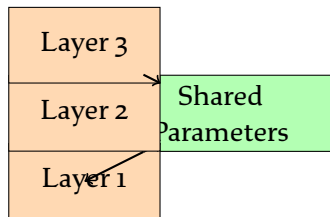


Parameters:

- $12 \times (\text{Attention} + \text{FFN})$

ALBERT (Sharing):

same parameters
All layers use



Parameters:

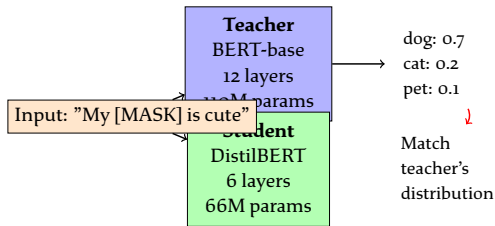
- $1 \times (\text{Attention} + \text{FFN})$

DistilBERT: Knowledge Distillation ⚡

Key idea: Train small model to mimic large model

Knowledge Distillation Process (Sanh et al. 2019):

1. **Teacher Model:** Large, pre-trained BERT
2. **Student Model:** Smaller model (6 layers instead of 12)
3. **Training Objective:**
 - Student learns to match teacher's output distributions
 - Not just hard labels, but soft probabilities
 - Also uses MLM loss on original task



Reference: Sanh et al. (2019) - "DistilBERT, a distilled version of BERT"

Significant efficiency gains!

Metric	BERT-base	DistilBERT	Ratio
Parameters	110M	66M	40% smaller
Inference Speed	1x	1.6x	60% faster
GLUE Score	79.6	77.0	97% retained

Performance on Specific Tasks:

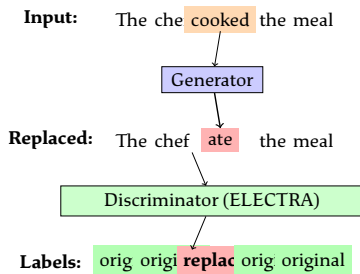
Task	BERT	DistilBERT
SQuAD 1.1 (F1)	90.9	86.9
SST-2 (Acc)	94.9	92.7
MNLI (Acc)	86.7	82.2

ELECTRA: Efficient Learning

Key idea: Learn from all tokens, not just 15%

ELECTRA's Approach (Clark et al. 2020):

- **Replace Token Detection** instead of Masked LM
- Use a small generator to replace some tokens
- Discriminator learns to detect which tokens were replaced
- **Every token provides a training signal!**



Reference: Clark et al. (2020) - "ELECTRA: Pre-training Text Encoders as Discriminators"

ELECTRA Benefits

More efficient pre-training

Advantages:

1. Sample Efficiency

- Learn from all tokens (100%) vs only masked (15%)
- Reaches same performance with less data
- Faster convergence

2. Better Performance

- ELECTRA-Small outperforms BERT-Small
- ELECTRA-Base competitive with BERT-Large
- With same compute, ELECTRA is better

3. Computational Efficiency

- Smaller generator (1/4 to 1/2 size of discriminator)
- Faster to train than BERT
- Lower computational cost for same quality

Key Insight

Replace Token Detection is more sample-efficient than Masked LM because it provides a learning signal for every token!

Summary of key variants

Model	Key Innovation	Advantages	Best For
BERT	Masked Language Model, bidirectional	Original, well-studied, strong baseline	General understanding tasks
RoBERTa	Better training (dynamic masking, no NSP, more data)	Best performance, robust	When quality matters most
ALBERT	Parameter sharing, factorized embeddings	Fewer parameters, memory efficient	Limited memory/storage
DistilBERT	Knowledge distillation	Fast inference, small size	Production, edge deployment
ELECTRA	Replace token detection	Sample efficient, good performance	Limited pre-training compute

General Guidelines:

- **Best quality:** RoBERTa-Large
- **Best efficiency:** DistilBERT
- **Limited memory:** ALBERT

Other Notable BERT Variants

The BERT family keeps growing!

1. DeBERTa (Microsoft, 2020)

- Disentangled attention (separate content and position)
- Enhanced mask decoder
- State-of-the-art on SuperGLUE

2. ERNIE (Baidu, 2019)

- Entity-level and phrase-level masking
- Knowledge enhancement
- Strong on Chinese NLP tasks

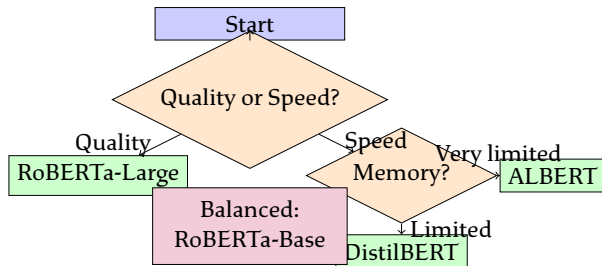
3. SpanBERT (Facebook, 2019)

- Mask random spans instead of random tokens
- Span boundary objective
- Better for span-based tasks (QA, coreference)

4. BART (Facebook, 2019)

- Encoder-decoder (not encoder-only)
- Denoising autoencoder with various corruptions
- Excellent for generation tasks

How to choose the right model for your task



Additional Considerations:

- **Domain:** Consider domain-specific pre-trained models (BioBERT, SciBERT, etc.)
- **Language:** Multilingual? Use mBERT, XLM-R
- **Task type:** Generation? Consider BART/T5 instead

Easy switching with HuggingFace

```
from transformers import AutoModel, AutoTokenizer

# BERT
bert_model = AutoModel.from_pretrained("bert-base-uncased")
bert_tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

# RoBERTa
roberta_model = AutoModel.from_pretrained("roberta-base")
roberta_tokenizer = AutoTokenizer.from_pretrained("roberta-base")

# ALBERT
albert_model = AutoModel.from_pretrained("albert-base-v2")
albert_tokenizer = AutoTokenizer.from_pretrained("albert-base-v2")
```

Discussion Questions

1. Training vs Architecture:

- RoBERTa shows training matters. Is architecture overrated?
- How much can we improve with just better training?
- What's the right balance?

2. Parameter Efficiency:

- ALBERT shares all layers. Why does this work?
- What are the limits of parameter sharing?
- Is there a "sweet spot"?

3. Knowledge Distillation:

- Why does student learn better from teacher than from labels?
- What information is in the soft probabilities?
- Can we distill even further?

4. Model Selection:

- How do you decide which model to use?
- Is it worth fine-tuning multiple variants?
- What about model ensembles?

Today we learned:

- RoBERTa: Better training matters
- ALBERT: Parameter sharing for efficiency
- DistilBERT: Knowledge distillation
- ELECTRA: Replace token detection
- How to choose the right variant

Next lecture (Lecture 17 - Applications of Encoder Models):

- Real-world BERT applications
- Classification, NER, QA, and more
- Cognitive neuroscience connections
- Understanding vs pattern matching debate
- Limitations and future directions
- Practical deployment tips

From theory to practice! 

Key Takeaways:

1. RoBERTa

- Training procedure matters as much as architecture
- Remove NSP, dynamic masking, more data = better results

2. ALBERT

- Parameter sharing dramatically reduces model size
- Factorized embeddings for efficiency
- 89% fewer parameters than BERT

3. DistilBERT

- Knowledge distillation for deployment
- 40% smaller, 60% faster, 97% performance

4. ELECTRA

- Replace token detection more sample-efficient
- Learn from all tokens, not just 15%

5. Model Selection

- Choose based on constraints (quality, speed, memory)
- HuggingFace makes it easy to experiment

Essential Papers:

- **Liu et al. (2019)** - "RoBERTa: A Robustly Optimized BERT Pretraining Approach"
- **Lan et al. (2019)** - "ALBERT: A Lite BERT for Self-supervised Learning"
- **Sanh et al. (2019)** - "DistilBERT, a distilled version of BERT"
- **Clark et al. (2020)** - "ELECTRA: Pre-training Text Encoders as Discriminators"
- **He et al. (2020)** - "DeBERTa: Decoding-enhanced BERT with Disentangled Attention"

Resources:

- HuggingFace Model Hub: <https://huggingface.co/models>
- Papers With Code: BERT variants leaderboard
- Model Cards: Detailed documentation for each variant

Discussion Time

Topics for discussion:

- BERT variants and improvements
- Model selection strategies
- Knowledge distillation
- Parameter efficiency
- Implementation questions

Thank you! 

Next: Applications and Real-World Use Cases!